nvidia.github.io

Multimodal Data — NVIDIA Generative AI Examples 0.5.0 documentation

5–6 minutes

This documentation is not the latest for generative-ai-examples. generative-ai-examples 0.7.0 is the latest version.

- Example Features
- Limitations
- Prerequisites
- Get an API Key for the Mixtral 8x7B Instruct API Endpoint
- Build and Start the Containers
- Next Steps

Example Features

This example deploys a developer RAG pipeline for chat Q&A and serves inferencing from NVIDIA API Catalog endpoints instead of NVIDIA Triton Inference Server, a local Llama 2 model, or local

GPUs.

Developers get free credits for 10K requests to any of the available models.

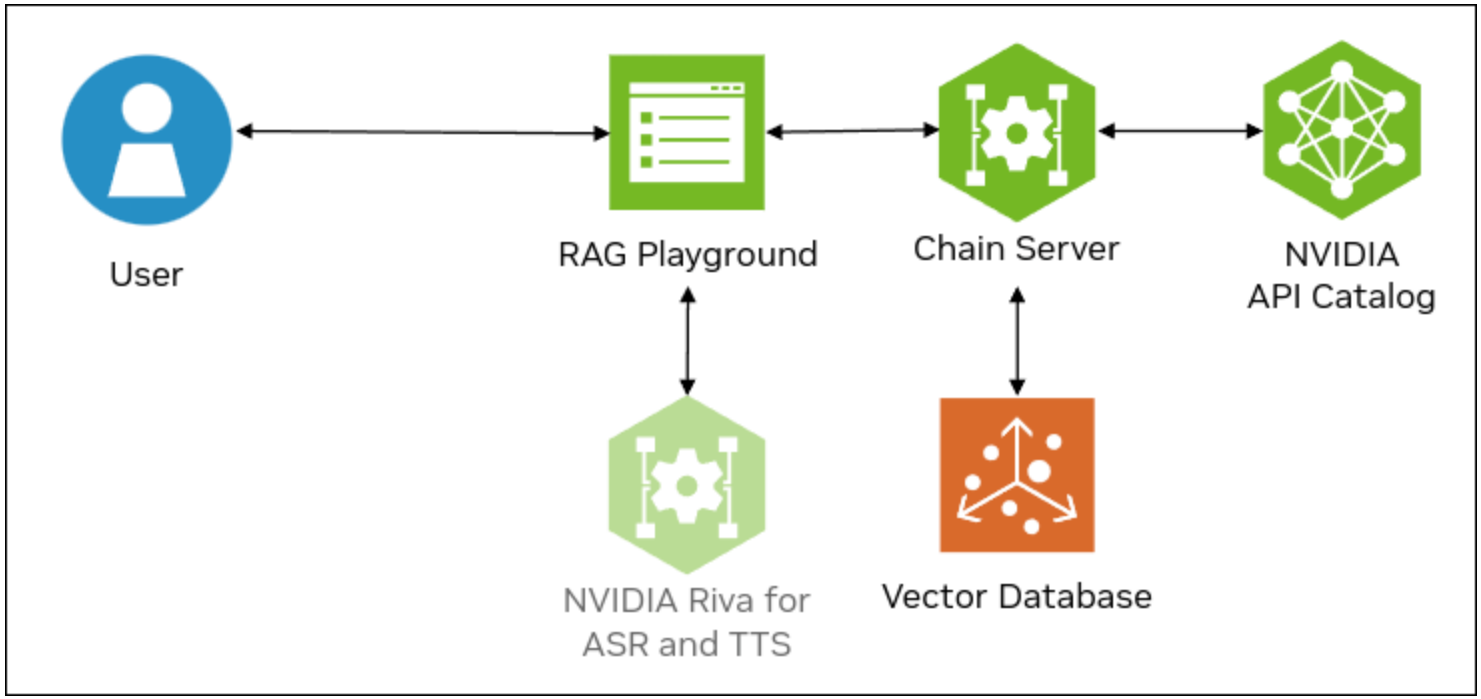
The key difference from the Using the NVIDIA API Catalog example is that this example demonstrates how work with multimodal data. The model works with any kind of image in PDF, such as graphs and plots, as well as text and tables.

This example uses models from the NVIDIA API Catalog.

Model	Embedding	Framework	Description	Multi-GPU	TRT-LLM	Model Location	Triton	Vecto Databa
ai-mixtral-8x7b-instruct for response generation ai-google-Deplot for graph to text conversion ai-Neva-22B for image to text conversion	nvolveqa_40k	Custom Python	QA chatbot	NO	NO	API Catalog	NO	Milvus

The following figure shows the sample topology:

- The sample chat bot web application communicates with the chain server. The chain server sends inference requests to NVIDIA API Catalog endpoints.
- Optionally, you can deploy NVIDIA Riva. Riva can use automatic speech recognition to transcribe your questions and use text-to-speech to speak the answers aloud.



Limitations

Although the AI Foundation Models endpoint uses the Neva_22B model for processing images, this example supports uploading images that are part of PDF files only. For example, after deploying the services, you cannot upload a PNG, JPEG, TIFF, or any other image format file.

Prerequisites

- Clone the Generative AI examples Git repository using Git LFS:

  ```
  $ sudo apt -y install git-lfs
  $ git clone git@github.com:NVIDIA/GenerativeAIExamples.git
  $ cd GenerativeAIExamples/
  $ git lfs pull
  ```

- Install Docker Engine and Docker Compose. Refer to the instructions for Ubuntu.
- Optional: Enable NVIDIA Riva automatic speech recognition (ASR) and text to speech (TTS).
- To launch a Riva server locally, refer to the Riva Quick Start Guide.
- In the provided `config.sh` script, set `service_enabled_asr=true` and `service_enabled_tts=true`, and select the desired ASR and TTS languages by adding the appropriate language codes to `asr_language_code` and `tts_language_code`.
- After the server is running, assign its IP address (or hostname) and port (50051 by default) to `RIVA_API_URI` in `deploy/compose/compose.env`.
- Alternatively, you can use a hosted Riva API endpoint. You might need to obtain an API key and/or Function ID for access.

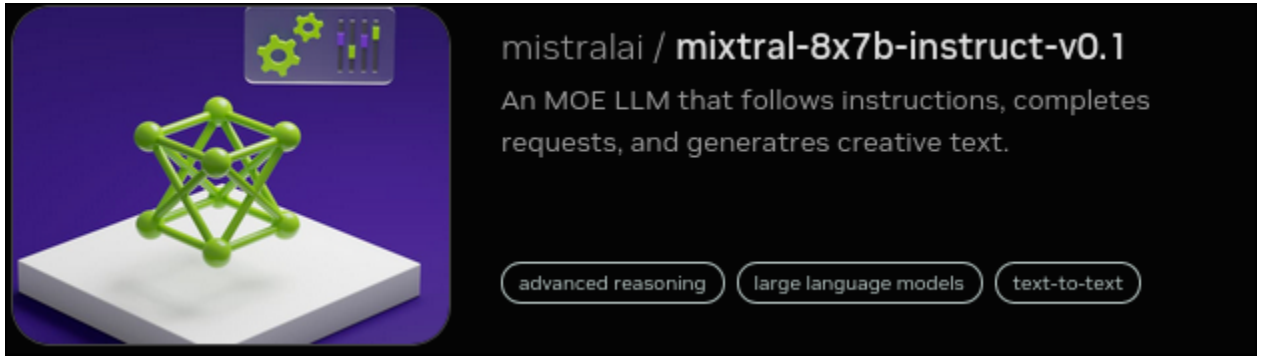
  In `deploy/compose/compose.env`, make the following assignments as necessary:

  ```
  export RIVA_API_URI="<riva-api-address/hostname>:<port>"
  export RIVA_API_KEY="<riva-api-key>"
  export RIVA_FUNCTION_ID="<riva-function-id>"
  ```

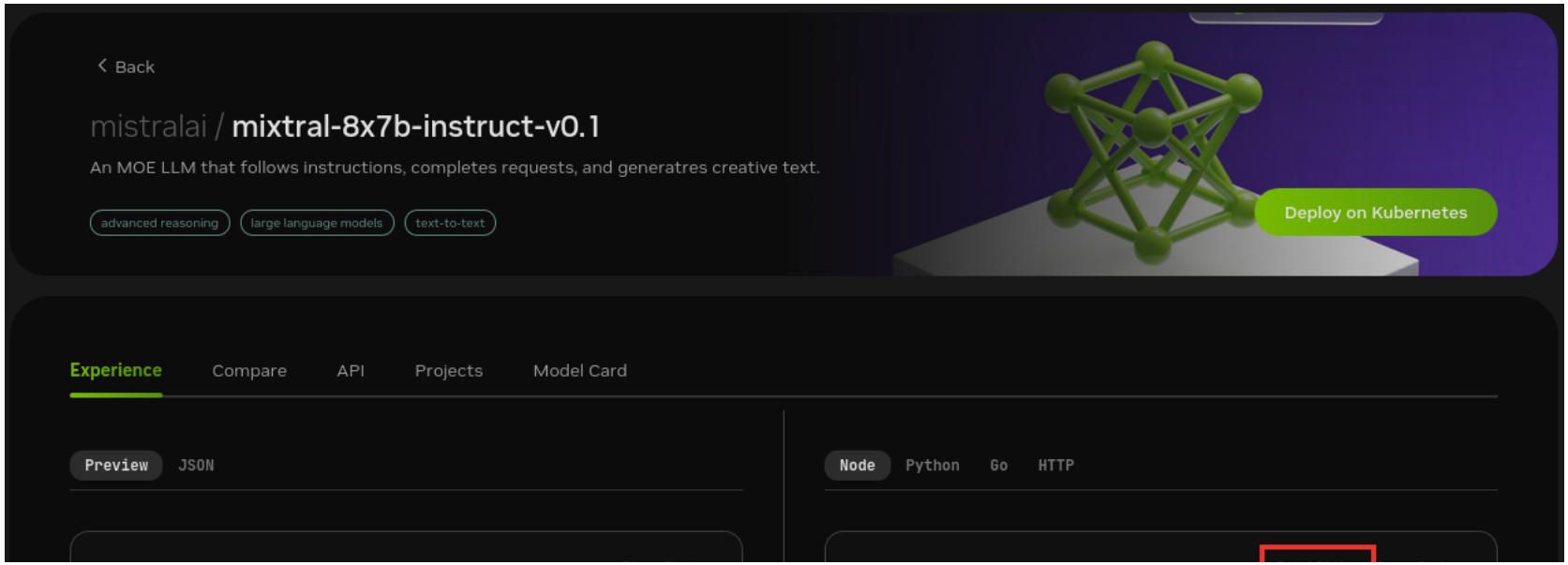
Get an API Key for the Mixtral 8x7B Instruct API Endpoint

Perform the following steps if you do not already have an API key. You can use different model API endpoints with the same API key.

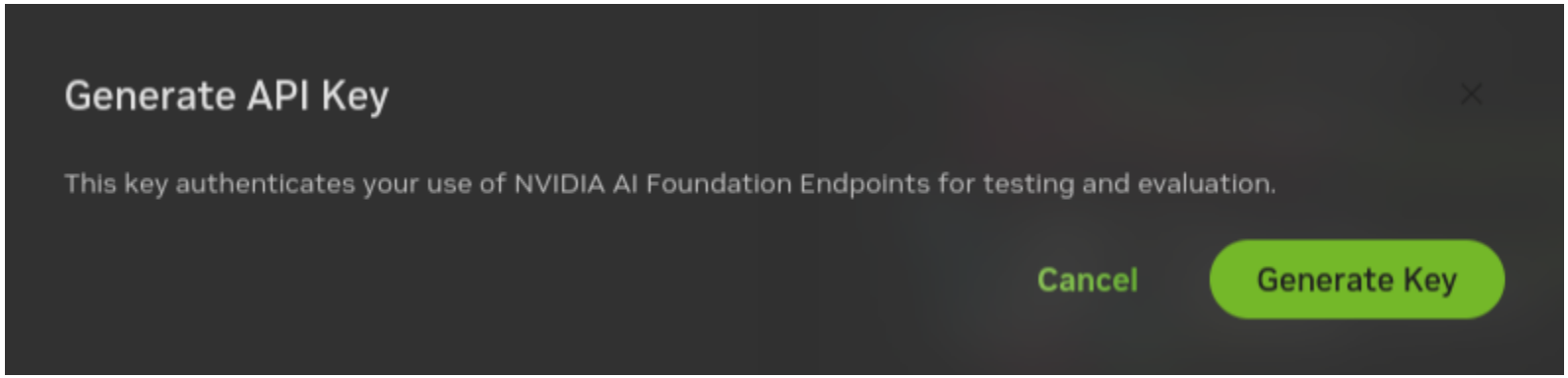
1. Navigate to https://build.nvidia.com/explore/discover.
2. Find the **Mixtral 8x7B Instruct** card and click the card.



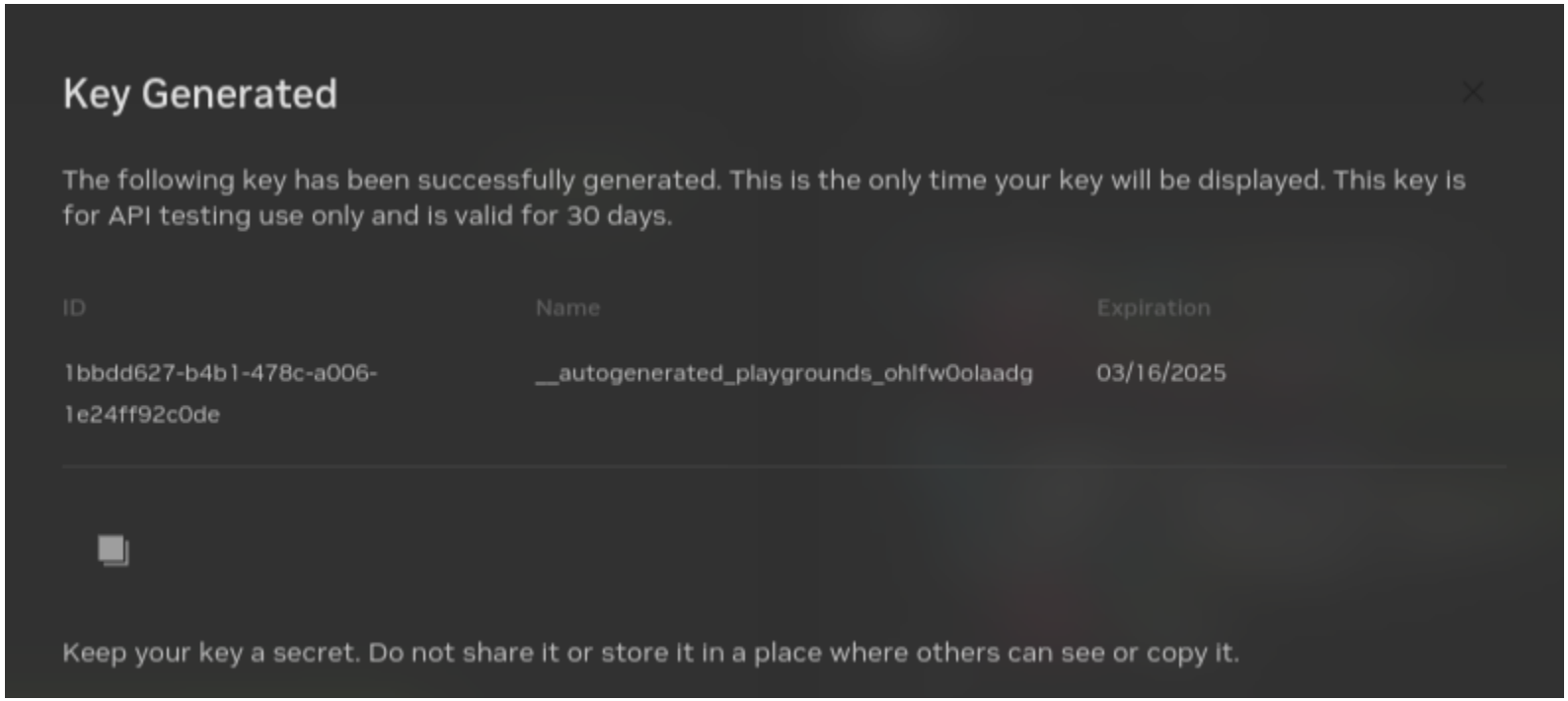
3. Click **Get API Key**.



4. Click **Generate Key**.



5. Click **Copy Key** and then save the API key. The key begins with the letters nvapi-.



Close Copy Key

Build and Start the Containers

1. In the Generative AI examples repository, edit the `deploy/compose/compose.env` file.

   Add the API key for the model endpoint:

   export NVIDIA_API_KEY="nvapi-..."

2. From the root of the repository, build the containers:

   $ docker compose --env-file deploy/compose/compose.env -f deploy/compose/rag-app-multimodal-chatbot.yaml build

3. Start the containers:

   $ docker compose --env-file deploy/compose/compose.env -f deploy/compose/rag-app-multimodal-chatbot.yaml up -d

   Example Output

   ✔ Network nvidia-rag Created
   ✔ Container chain-server Started
   ✔ Container rag-playground Started

4. Start the Milvus vector database:

   $ docker compose --env-file deploy/compose/compose.env -f deploy/compose/docker-compose-vectordb.yaml up -d milvus

Example Output

```
✔ Container milvus-minio      Started
✔ Container milvus-etcd       Started
✔ Container milvus-standalone  Started
```

5. Confirm the containers are running:

   ```
   $ docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Status}}"
   ```

   Example Output

   ```
   CONTAINER ID   NAMES            STATUS
   37dcdb4ffcb0   rag-playground     Up 3 minutes
   39718f6a2a06   chain-server       Up 3 minutes
   68af1e4dfb44   milvus-standalone   Up 2 minutes
   522b12ec17f0   milvus-minio       Up 2 minutes (healthy)
   ed48988c5657   milvus-etcd        Up 2 minutes (healthy)
   ```

Next Steps

- Access the web interface for the chat server. Refer to Using the Sample Chat Web Application for information about using the web interface.
- Upload one or more PDF files with graphics, plots, and tables.
- Enable the **Use knowledge base** checkbox when you submit a question.
- Stop the containers by running `docker compose -f deploy/compose/rag-app-multimodal-chatbot.yaml down` and `docker compose -f deploy/compose/docker-compose-vectordb.yaml down`.